

## à la découverte de JPA

Les objectifs de ce travail pratique sont les suivants :

- Comprendre les mécanismes de JPA
- Réaliser une application en utilisant JPA en se plaçant dans un cadre classique de développement sans serveur d'application au départ.

**Recommandations.** Ce TP utilise des technologies abordées en cours d'un point de vue théorique, mais la documentation technique peut être facilement trouvée sur internet.

Quelques exemples de sites qui fournissent de la documentation sur JPA sont les suivantes:

[https://en.wikibooks.org/wiki/Java\\_Persistence](https://en.wikibooks.org/wiki/Java_Persistence)

<http://www.oracle.com/technetwork/middleware/ias/toplink-jpa-annotations-096251.html>

Être autodidacte est une compétence essentielle pour tout informaticien; n'hésitez pas à chercher des tutoriels si vous êtes bloqués.

## Sujet : Application de Vente de Tickets de Concert en Ligne

### 1. Introduction

#### 1.1 Contexte

L'objectif de ce projet est de développer une application web et mobile permettant aux utilisateurs d'acheter des tickets de concert en ligne. L'application doit offrir une expérience utilisateur fluide, sécurisée et intuitive, tout en permettant aux organisateurs de concerts de gérer simplement leurs événements.

#### 1.2 Objectifs

- Permettre aux utilisateurs de rechercher, consulter et acheter des tickets de concert.
- Offrir une plateforme de gestion pour les organisateurs d'événements.
- Assurer la sécurité des transactions et des données utilisateurs.
- Proposer une interface responsive et accessible sur un navigateur web.

---

#### 1.3. Fonctionnalités

## Fonctionnalités pour les Utilisateurs

- **Recherche et Consultation des Concerts :**
  - Recherche de concerts par artiste, lieu, date, genre musical.
  - Affichage des détails de l'événement (date, lieu, prix, description).
  - Filtres et tris (par prix, date, popularité).
- **Achat de Tickets :**
  - Sélection des places (si applicable).
  - Envoi du ticket par email ou téléchargement direct.
  - Option d'annulation et de remboursement (selon les conditions de l'organisateur).
- **Gestion des Tickets :**
  - Accès à un espace personnel pour consulter les tickets achetés.
  - Possibilité de transférer un ticket à un autre utilisateur.
- **Notifications :**
  - Alertes pour les nouveaux concerts, rappels d'événements, et offres spéciales.

## Fonctionnalités pour les Organismes

- **Gestion des Événements :**
  - Création, modification et suppression d'événements.
  - Ajout de détails (date, lieu, description, prix, capacité).
  - Gestion des stocks de tickets.
- **Statistiques et Rapports :**
  - Suivi des ventes en temps réel.
  - Rapports sur les performances des événements.
- **Communication :**
  - Envoi de notifications aux acheteurs de tickets (changements d'horaire, annulations, etc.).

### 2.3 Fonctionnalités Administrateur.

- **Gestion des Événements :**
    - Validation des événements créés par les organismes.
    - Modération des contenus.
- 

## 1.4 Exigences Techniques

### Plateformes

- **Web :** Compatibilité avec les navigateurs modernes (Chrome, Firefox, Safari, Edge).

### Technologies

- **Front-end :** HTML5, CSS3, JavaScript (React.js ou Angular).
- **Back-end :** Java (JPA)

- **Base de données** : MySQL, PostgreSQL.
  - **Paiement** : Intégration d'API de paiement (Stripe, PayPal, etc.) (optionnelle ;).
- 

## 1.5 Design et Expérience Utilisateur (UX/UI)

### Design

- Interface moderne et épurée.
- Palette de couleurs et typographie cohérentes avec la marque.
- Expérience utilisateur intuitive et accessible.

### Responsive Design

- Adaptation de l'interface à tous les dispositifs (desktop, mobile, tablette).
- Temps de chargement optimisé.

Dans une première étape, nous nous intéressons uniquement au modèle métier de cette application. Faites un premier diagramme de classes sur feuille blanche de ce modèle métier, créer les classes Java correspondantes et utilisons JPA pour créer la base de données et la couche d'accès aux données.

### Organisation.

Vous trouverez un template de projet sur <https://github.com/barais/tpjpa2023>

Vous pouvez forker le projet contenant :

- Un template de projet pour la construction d'application autonome utilisant JPA, hibernate et hsqldb.

Décompressez ce projet sur votre compte et importez dans eclipse ou intelliJ.

### Pour eclipse 4.X

Depuis eclipse 4.X, le support de maven s'est amélioré. Pour importer votre projet. File -> import -> maven -> existing maven project.  
Votre projet est configuré.

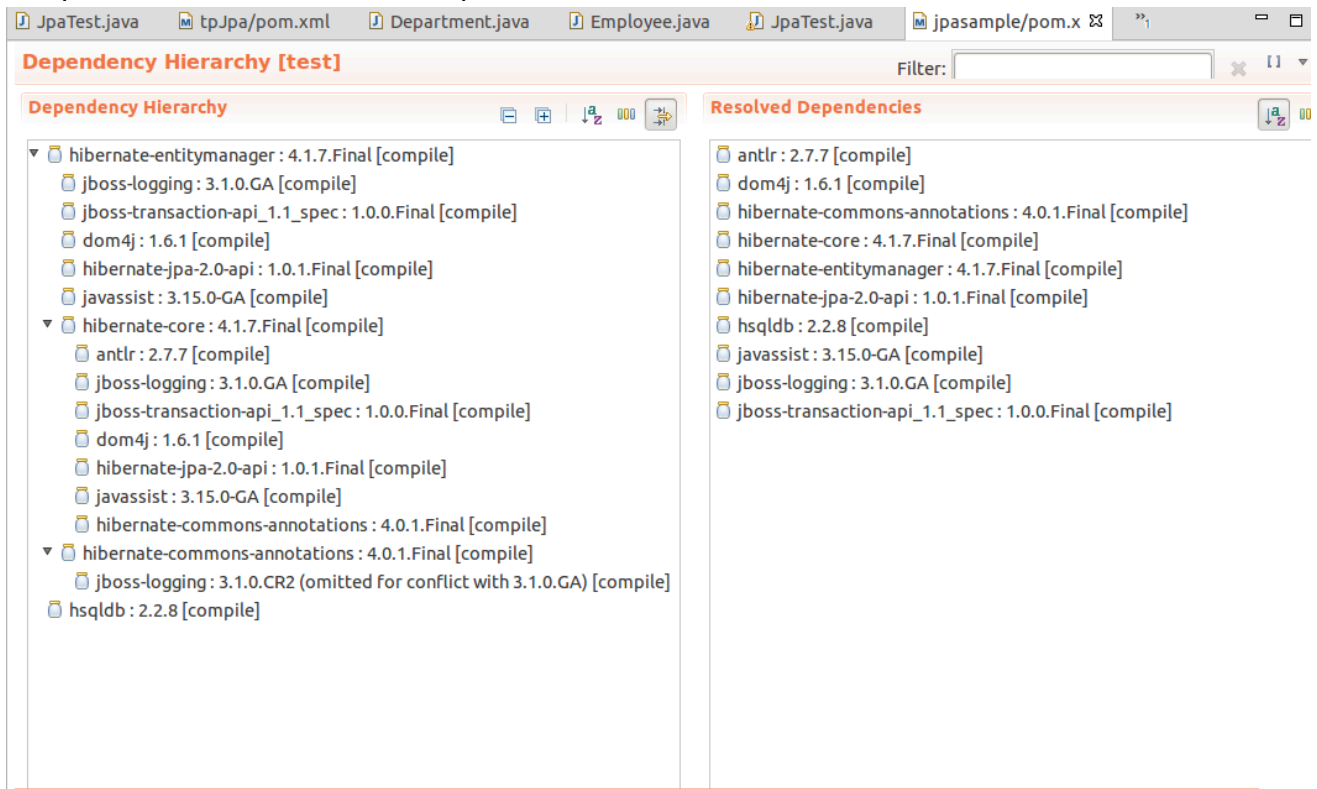
**Démarrage de la base de données.** Puis dans la version copiée sur votre compte allez dans le répertoire du projet. Vous trouverez là le script de démarrage de la base de données (run-hsqldb-server.sh) et le script du démarrage du Manager (show-hsqldb.sh). Lancez le système de base de données, puis le *Manager*. Connectez vous à la base de données (login : sa – et pas de mot de passe : -- URL de connexion : jdbc:hsqldb:hsqldb://localhost/ ). Le fichier de données se trouve dans le répertoire

Data. Vous pourrez supprimer l'ensemble des fichiers de ce répertoire si vous souhaitez réinitialiser complètement votre système de base de données.

## Question 0.

Regardez rapidement le pom.xml, vous constaterez que c'est un projet simple avec deux dépendances (hibernate et hsqldb (driver jdbc pour hsqldb)).

Ce qui donne finalement les dépendances suivantes.



## Question 1.

Transformez une première classe en entité.

Travaillez uniquement sur le champ id et les attributs de la classe. Créez plusieurs instances de cette classe. Rendez ces instances persistantes. Regardez dans le manager de la base de données le résultat.

Vous devez obtenir ce genre de code **mais sur vos classes métier**.

```
package domain;
```

```
import jakarta.persistence.Entity;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.Id;  
import jakarta.persistence.ManyToOne;
```

```
@Entity
```

```
public class Employee {  
    private Long id;
```

```

private String name;

private Department department;

public Employee() {
}

public Employee(String name, Department department) {
    this.name = name;
    this.department = department;
}

public Employee(String name) {
    this.name = name;
}

@Id
@GeneratedValue
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

@ManyToOne
public Department getDepartment() {
    return department;
}

public void setDepartment(Department department) {
    this.department = department;
}

@Override
public String toString() {
    return "Employee [id=" + id + ", name=" + name + ", department="
        + department.getName() + "];"
}

}

```

Question 2.

Même travail avec une première association entre deux entités.

Vous devez obtenir un code qui ressemble à cela mais sur vos classes métier. N'oubliez pas de remplacer les attributs mis à *Transient* sur vos classes précédentes.

```
package test.testjpa.domain;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
import javax.persistence.CascadeType;
```

```
import javax.persistence.Entity;
```

```
import javax.persistence.GeneratedValue;
```

```
import javax.persistence.Id;
```

```
import javax.persistence.OneToMany;
```

```
@Entity
```

```
public class Department {
```

```
    private Long id;
```

```
    private String name;
```

```
    private List<Employee> employees = new ArrayList<Employee>();
```

```
    public Department() {
```

```
        super();
```

```
    }
```

```
    public Department(String name) {
```

```
        this.name = name;
```

```
    }
```

```
@Id
```

```
@GeneratedValue
```

```
    public Long getId() {
```

```
        return id;
```

```
    }
```

```
    public void setId(Long id) {
```

```
        this.id = id;
```

```
    }
```

```
    public String getName() {
```

```
        return name;
```

```
    }
```

```
    public void setName(String name) {
```

```
        this.name = name;
```

```
    }
```

```

    @OneToMany(mappedBy = "department", cascade = CascadeType.PERSIST)
    public List<Employee> getEmployees() {
        return employees;
    }

    public void setEmployees(List<Employee> employees) {
        this.employees = employees;
    }
}

```

### Question 3.

Finissez le modèle métier présenté précédemment. Intégrez une classe de service permettant de peupler la base mais aussi de faire des requêtes sur la base de données.

Le code pour créer les entités peut ressembler à cela :

```

package test.testjpa.jpa;

import java.util.List;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.EntityTransaction;
import javax.persistence.Persistence;

import test.testjpa.domain.Employee;
import test.testjpa.domain.Department;

public class JpaTest {

    private EntityManager manager;

    public JpaTest(EntityManager manager) {
        this.manager = manager;
    }

    /**
     * @param args
     */
    public static void main(String[] args) {
        EntityManagerFactory factory =
            Persistence.createEntityManagerFactory("example");
        EntityManager manager = factory.createEntityManager();
        JpaTest test = new JpaTest(manager);

        EntityTransaction tx = manager.getTransaction();
        tx.begin();
        try {
            test.createEmployees();
        } catch (Exception e) {

```

```

        e.printStackTrace();
    }
    tx.commit();

    test.listEmployees();

    manager.close();
    System.out.println(".. done");
}

private void createEmployees() {
    int numOfEmployees = manager.createQuery("Select a From Employee a",
Employee.class).getResultList().size();
    if (numOfEmployees == 0) {
        Department department = new Department("java");
        manager.persist(department);

        manager.persist(new Employee("Jakab Gipsz",department));
        manager.persist(new Employee("Captain Nemo",department));

    }
}

private void listEmployees() {
    List<Employee> resultList = manager.createQuery("Select a From Employee a",
Employee.class).getResultList();
    System.out.println("num of employess:" + resultList.size());
    for (Employee next : resultList) {
        System.out.println("next employee: " + next);
    }
}
}

```

#### Question 4. Connexion à une base mysql

Modifiez le fichier *persistence.xml* afin de vous connecter sur une base de données MySQL de l'istic.

Pour ce faire connecter vous sur <http://anteros.istic.univ-rennes1.fr> (besoin du vpn <https://istic.univ-rennes1.fr/intranet/services#section-4>) pour vous créer votre base de données. Puis en vous inspirant de l'exemple suivant <http://snipplr.com/view/4450/> modifiez votre fichier *persistence.xml* pour vous connecter à votre base de données. Il est aussi nécessaire d'ajouter le driver jdbc vers mysql. Pour ce faire ajoutez la dépendance dans le *pom.xml*

Si vous vous connecter à l'IISTIC (anteros)

```

<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>5.1.49</version>
</dependency>

```

Si vous vous connectez à votre mysql installé sur votre machine.

```

<dependency>

```



```
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>8.0.22</version>
</dependency>
```

**Question 5. Portez votre application et gérer au minimum une relation d'héritage, les requêtes, une requête nommée.**

Pour l'héritage, on peut imaginer des "tickets" de type différents, par exemple des tickets premium ou des tickets "last minute" sans place réservée. Tester différentes stratégies d'héritage et regarder l'impact sur la structure de votre base de donnée relationnelle.

Faites vos requêtes en utilisant les criteria query.

<http://stackoverflow.com/questions/5705291/select-in-equivalent-in-jpa2-criteria>

**Question 6. Mise en évidence du problème de n+1 select.**

Pour comprendre le problème du n+1 select.

récupérez le code joint depuis github.

<https://github.com/barais/tpM2s.git>

Comme d'habitude c'est un projet Maven.

Lancez JPATest pour peupler la base de données.

Lancez N1select pour faire une requête en chargement paresseux (problème du n+1 select)

Lancez JoinFetch pour faire une requête en chargement au plus tôt (sans le problème du n+1 select)

Comparez les performances et le nombre de requêtes réellement effectuées.

Bon TP

Adrien et Olivier

---